Crawl, Walk, Run

Applying the FinOps Maturity Model to an LLM Product

[Your Name]

SOFTCON 2026

I'm Cy 👋



NOW

AI software developer at Cambridge University Press & Assessment

I TALK ABOUT

AI and agents

BEFORE

Taught data science and analytics

OFF THE CLOCK

- 🥁 Drums
- 🎮 Pokémon
- 🎮 Monster Hunter
- ✈️ Travel
- 🥾 Hiking

WHERE IT GOES WRONG

Two ways to get LLM cost wrong

Too early

Build cost dashboards and budgets for a prototype.

It might be deleted next week.

Too late

Ship to production, see the bill, then switch to a cheaper model.

Without checking what that does to quality.

Both ignore how mature the product is.

THE QUESTION FOR THE NEXT 50 MINUTES

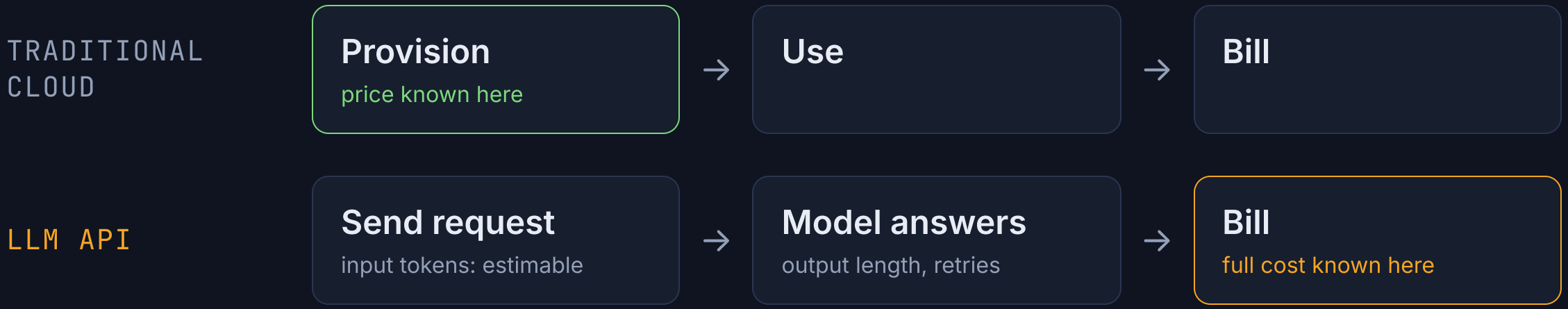
Is your unit cost falling as adoption rises, or is every new user making the economics worse?

01

Why LLM cost is different

THE CORE INVERSION

With LLMs you learn the full cost after the work is done



That makes forecasting, budgeting, anomaly detection and allocation harder.

WHY THE BUSINESS CARES

Margins are lower, and cheaper tokens do not fix that



Inference is about 23% of revenue at scaling-stage AI companies.

Every model call is a per-use cost, so it counts as cost of goods sold.

Token prices fall about 10x a year, but cheaper calls get used in more places, so total spend can still rise.

Sources: ICONIQ State of AI (Jan 2026); a16z, "The New Business of AI" and "LLMflation." Survey and self-reported data, not audited.

THE FRAMEWORK

FinOps for AI: Crawl, Walk, Run

CRAWL

Prototype, MVP or pilot. Fail fast, with cost and time limits set up front. Budgets are revised often.

WALK

Used in real business processes. Basic automated cost tracking and basic anomaly analysis.

RUN

Core to the business. Advanced anomaly tracking and ROI. Budgets stay fairly constant.

The paper is written for FinOps practitioners. This talk covers what it means for engineers.

Source: FinOps Foundation, "FinOps for AI Overview" (CC BY 4.0).

THE SOURCES

Three papers, one pipeline

FinOps for AI Overview

What and when. The three phases, 10 KPIs, and the optimization levers.

Gives this talk its structure.

Cost and Usage Tracker

How to measure. Log actual tokens per call, with a request id, a dated price table, and one key per use case.

Our wrapper in Crawl and Walk.

Cost Estimation

How to forecast. Tokens per request × volume × price, then compare with actuals. Target: over 90% accuracy.

Our estimate in Crawl.

We apply all three to one app and report what held up and what did not.

FinOps Foundation working-group papers. Links on the sources slide.

WHAT I BUILT

One small app, taken through all three phases

What it does

Suki Inbox: triage for a small online shop's Taglish Messenger inbox. Classify, extract order details, draft a reply that follows shop policy.

Models

Ollama Cloud. `gpt-oss:120b` ($0.15 / $0.60 per 1M tokens) and `gpt-oss:20b` ($0.07 / $0.30).

Traffic

60 hand-labeled synthetic tickets, 3 calls each. Several recorded runs per config.

Code

`github.com/mattekudacy/softcon-demo`

A reference implementation, not a production system. Numbers are real, traffic is synthetic.

02

Crawl

Is this idea worth building?

CRAWL

In Crawl, mostly ignore cost

- Goal: find out quickly and cheaply whether the idea works.
- Set a time limit and a spend limit up front. Then stop worrying about cost.
- Do spend on whatever you are testing. Usually that is output quality.
- Quality and user feedback tell you more than dollars at this stage.

CRAWL

Minimum viable instrumentation

```
def call_llm(feature, model, messages):
    resp = client.generate(model=model, messages=messages)
    log.info("llm_call", extra={
        "request_id": new_id(),
        "feature": feature,
        "model": model,
        "input_tokens": resp.usage.input_tokens,
        "output_tokens": resp.usage.output_tokens,
        "cost_usd": price(model, resp.usage),  # dated price table
    })
    return resp
```

No dashboard yet. Log every call with a `feature` tag, using the token counts the provider returns. The Tracker paper ranks request counting as least accurate, then token estimates, then actual counts. Adding the tag later means editing every call site.

WHAT I MEASURED · CRAWL

The naive version

COST / TICKET	TOKENS / TICKET	QUALITY SCORE
$0.00067	3,171	0.959
$0.00022 per call		

Every request goes to one model. No caching. Full context every time. Mean of 3 recorded runs, replayed offline. A ticket is 3 calls.

A COST GUESS VS THE MEASURED COST (ACCURACY)



Input tokens came within 1%. Output was 2.2x the guess, because reasoning tokens bill as output.

COMMON CRAWL MISTAKE

Building Run-level governance for something that might be deleted next week.

03

Walk

Real users now. Someone has to own the cost.

WALK

Move from total spend to unit cost

Cost per inference

`total inference cost ÷ requests`

FinOps KPI

Cost per feature

`spend grouped by feature tag`

Ours: the allocation problem

Cost per user

`spend ÷ active users`

Ours: the allocation problem

Rising total spend can be healthy. Rising unit cost is the warning sign.

Cost per inference: FinOps Foundation, "FinOps for AI Overview" (one of its 10 KPIs). Per feature and per user are our breakdowns of it, answering the paper's allocation challenge.

WALK

Cost is only half of ROI

What the Overview gives you

`ROI = (benefits - costs) ÷ costs`

Also tracked: time to business value, against your forecast.

Six value pillars

Cost efficiency, resilience, user experience, productivity, sustainability, business growth.

Pick the ones your feature actually moves.

Everything else in this talk is the cost side. The paper lists the benefit pillars but not how to measure them. Decide how you will, before launch.

FinOps Foundation, "FinOps for AI Overview": ROI and time-to-value KPIs, value pillars.

WALK

Budgets and alerts

Token budget per feature

Set it when you design the feature, like a performance budget. A target cost per request belongs in the spec. Give each use case its own API key, as the Tracker paper advises.

Anomaly alerts

Compare against your own baseline. If tokens per request double, find out why that day. Re-check your estimate whenever average request size changes (Cost Estimation paper).

Common causes: retry loops, history that keeps growing, a longer prompt template.

Also in Walk: re-run your evals whenever a prompt or request size changes, and track quality per feature next to cost. Our addition, not from the FinOps papers.

WHAT I MEASURED · WALK

What the breakdown showed

SHARE OF CALLS



SHARE OF SPEND



WHERE ONE CLASSIFY CALL'S ~1,000 TOKENS GO



shop policy, sent every call · ticket and instructions ~125 · answer: 60 tokens

`classify` and `extract` made up **57%** of spend, to return a one-word label or a small JSON object.

Mean of 3 recorded runs. Policy size estimated at 4 characters per token.

04

Run

The business depends on it. Optimize without breaking it.

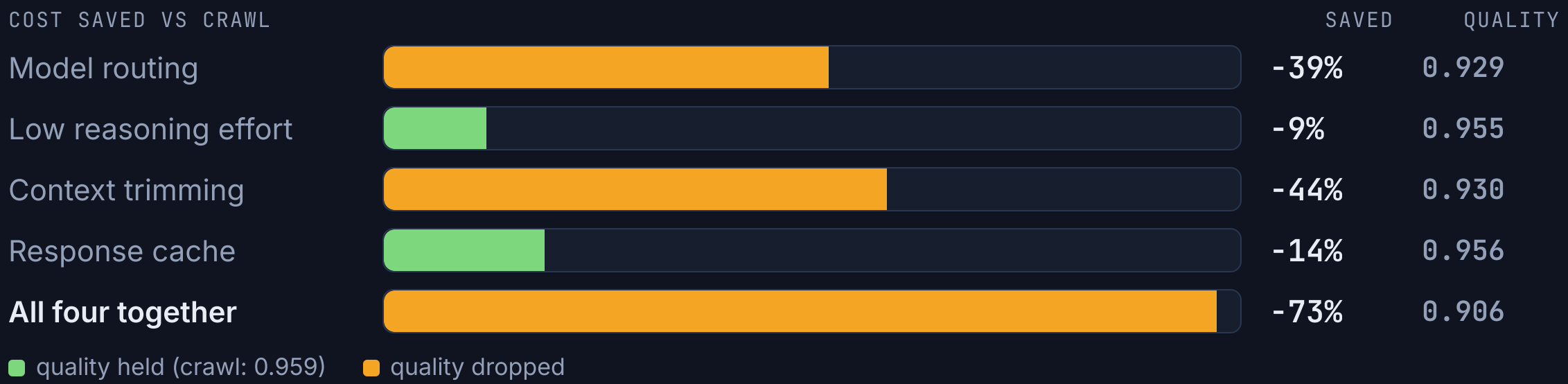
RUN

Cut spend that changes nothing first. Protect spend that holds quality and latency.

The Overview adds two cautions. Integration costs get higher priority and are less likely to be optimized. And total cost should not fall below a baseline relative to the benefits.

RUN

Four levers, measured



Each lever alone, then all four. They add up to 106%, so they overlap.

Routing, trimming and caching are in the FinOps paper. Reasoning effort is our addition. Batching is in the paper too, but Ollama Cloud has no batch API, so it is not measured. Single levers are the mean of 2 runs, all four of 3.

RUN

Paper vs. production

FRUGALGPT (STANFORD)

Up to 98% cost reduction

Matching the best single model, on research benchmarks.

OUR REFERENCE APP

39% cost reduction

Routing alone, on our mix of requests. Quality fell from 0.959 to 0.929.

The gap comes from your mix of requests. Routing saves more when more of them are easy.

Chen, Zaharia & Zou, "FrugalGPT," arXiv:2305.05176. See also RouteLLM (Ong et al.).

RUN

The cheapest model that passes your evals

- Using a large model for an easy task means paying for quality you do not use.
- You cannot safely switch to a cheaper model without evals, so build them first.
- Run the evals again when new models come out. Prices and quality both change.
- Try prompt changes before anything heavier. In the Cost Estimation paper's example, few-shot prompting matched a fine-tuned model for less money.
- Red team before you ship a cut. Cheaper models and trimmed context can fail on odd or hostile inputs that an average score hides. Our addition.

"Model choice quality alignment" KPI, FinOps Foundation, "FinOps for AI Overview."

WHAT I MEASURED · RUN

Same app, after optimization

METRIC	CRAWL BASELINE	RUN	CHANGE
Cost / ticket	$0.00067	$0.00018	−73%
Tokens / ticket	3,171	942	−70%
Quality score	0.959	0.906	−0.053
p95 time / ticket	6.9 s	10.0 s	+3.1 s

Cost and tokens fell a lot. Quality and speed got worse.

Mean of 3 runs. Time is wall-clock on a shared cloud endpoint, so treat it as rough. The smaller model was slower on replies here: about 4.6 to 5.5 s against 2.5 s.

RUN

What a month costs, and what the levers are worth

CONFIG	COST / TICKET	COST / MONTH	SAVED / MONTH	QUALITY
Crawl baseline	$0.00067	~$242	-	0.959
Run, all levers	$0.00018	~$66	~$176	0.906

`monthly cost = requests per month × tokens per request × price per token`

Volume is the Cost Estimation paper's own example: 1,000 requests an hour, 12 hours a day, 30 days, so 360,000 a month. Use your own.

Is $176 a month worth 0.05 of quality? Only your benefit number can answer that. A saving only counts if quality holds. The Overview says to weigh each saving against its negative effects. The Quality column is that check.

Method: FinOps Foundation, "Cost Estimation of AI Workloads." Report this per use case, as the Tracker paper's showback and chargeback describe.

05 · LIMITS

Where the framework breaks

Agents

No accepted way to split cost across multi-step and agent calls. The FinOps paper says so itself.

Forecasting

When users drive the bill, trend-based forecasts are unreliable early on.

Org structure

The model assumes a FinOps team. Many shops have none, and sell fixed-price projects on metered AI.

Bad outputs

No good benchmarks for failed or hallucinated outputs. Our reply check only counts required facts.

Billing and ownership

Vendor billing is inconsistent, and AI spend is spread across teams, so ownership is hard to assign.

TAKEAWAY

The checklist

PHASE	MEASURE	SKIP	ENFORCE
Crawl	Tokens, model, feature per call. Quality on a small hand-labeled set.	Dashboards, allocation, forecasts	A spend cap on the key
Walk	Cost per inference, feature, user. Quality per feature.	Aggressive optimization	Token budgets, anomaly alerts. Re-run evals when prompts change.
Run	Unit cost vs. adoption, quality per model	Cuts that hurt quality or latency	Routing, caching, evals as a gate, red teaming, cost in design review

Of the Overview's 10 KPIs, this app measures 5. Two do not apply to an API app (training cost, GPU utilization). Three need business data (ROI, time to value, time to first prompt).

Evals in Walk and red teaming in Run are our additions. The FinOps papers do not cover them.

BACK TO THE QUESTION

Is your unit cost falling as adoption rises?

If you cannot answer that yet, start with the phase you are in.

SOURCES

Read further

- FinOps Foundation, "FinOps for AI Overview." finops.org/wg/finops-for-ai-overview
- FinOps Foundation, "How to Build a Generative AI Cost and Usage Tracker" finops.org/wg/how-to-build-a-generative-ai-cost-and-usage-tracker
- FinOps Foundation, "Cost Estimation of AI Workloads" finops.org/wg/cost-estimation-of-ai-workloads
- Chen, Zaharia & Zou, "FrugalGPT." arXiv:2305.05176
- Ong et al., "RouteLLM." ICLR 2025
- Casado & Bornstein, "The New Business of AI." a16z
- a16z, "LLMflation"; ICONIQ, "State of AI" (Jan 2026)

Code and checklist: github.com/mattekudacy/softcon-demo

Questions?

[Your Name] · [contact / LinkedIn]

Repo: github.com/mattekudacy/softcon-demo